

Rapport d'Analyse Dynamique

MobSF – InsecureBankv2

Auteur	Chagdaly Hiba
Date	03 Mai 2026
Cours	Sécurité des applications mobiles
Lab	Lab 07 – Analyse Dynamique avec MobSF
Outil	MobSF v4.5.0 (Docker) + Frida
APK	InsecureBankv2.apk
Score	28/100 – CRITIQUE

*Ce rapport est réalisé dans un cadre pédagogique.
L'application analysée est volontairement vulnérable.*

Informations générales

Propriété	Valeur
Application	InsecureBankv2
Package	com.android.insecurebankv2
Version	1.0
Target SDK	22 (Android 5.1)
Min SDK	15 (Android 4.0)
SHA-256	b18af2a0e44d7634bbcdf93664d9c78a2695e050393fcfbb5e8b91f902d194a4
Score sécurité	28/100 – CRITIQUE
Trackers	3/432

TABLE 1 – Informations de l'APK analysé

Configuration de l'environnement d'analyse

2.1 Connexion ADB et démarrage MobSF

Avant de lancer MobSF, l'émulateur Genymotion doit être connecté via ADB. MobSF utilise la variable `MOBSF_ANALYZER_IDENTIFIER` pour cibler le bon device lors de l'analyse dynamique.

```
adb connect 169.254.79.101:5555
sudo docker run -it --rm -p 8000:8000 \
  -e MOBSF_ANALYZER_IDENTIFIER="169.254.79.101:5555" \
  opensecurity/mobile-security-framework-mobsf:latest
```

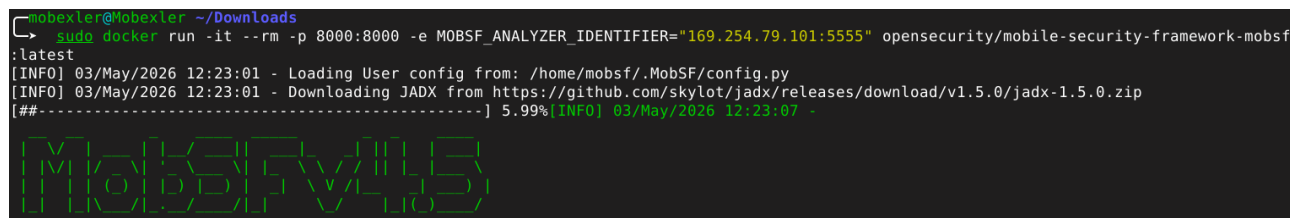


FIGURE 1 – Connexion ADB – device connecté sur 169.254.79.101 :5555

Observation : Le device Genymotion apparaît avec le statut **device**, confirmant que ADB communique avec l'émulateur. MobSF utilisera cette connexion pour installer l'APK et injecter Frida automatiquement.

```
mobexler@Mobexler ~/Downloads
└─> adb connect 169.254.79.101:5555
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 169.254.79.101:5555
mobexler@Mobexler ~/Downloads
└─> adb devices
List of devices attached
169.254.79.101:5555    device
```

FIGURE 2 – MobSF v4.5.0 démarré avec `MOBSF_ANALYZER_IDENTIFIER` configuré

Observation : MobSF démarre en téléchargeant JADX et en chargeant la configuration. La variable MOBSF_ANALYZER_IDENTIFIER garantit que MobSF cible le bon émulateur pour l'analyse dynamique.

Analyse statique préliminaire

Avant l'analyse dynamique, MobSF effectue une analyse statique pour établir une baseline des vulnérabilités.

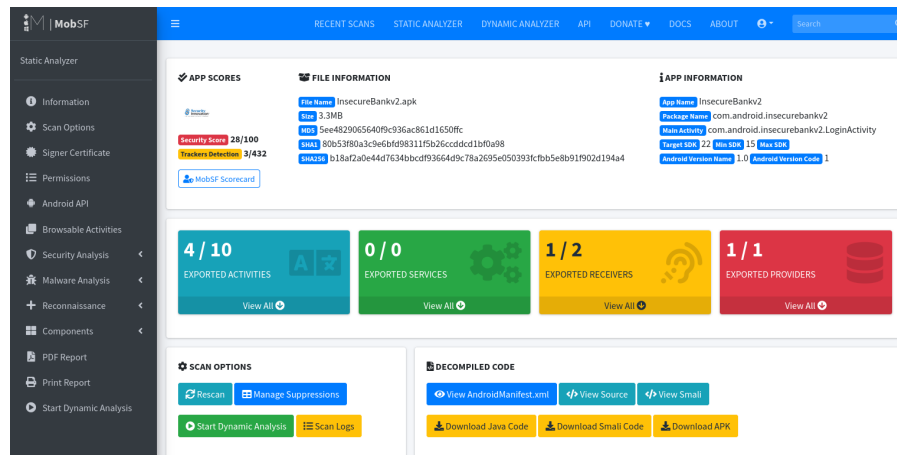


FIGURE 3 – Score de sécurité MobSF – 28/100 CRITIQUE

Observation : Le score de 28/100 est critique. On note 4 activités exportées sur 10, 1 receiver exposé sur 2, et 1 provider exposé – une surface d'attaque très large pour une application bancaire.

Analyse dynamique

4.1 Interface Dynamic Analyzer et Frida

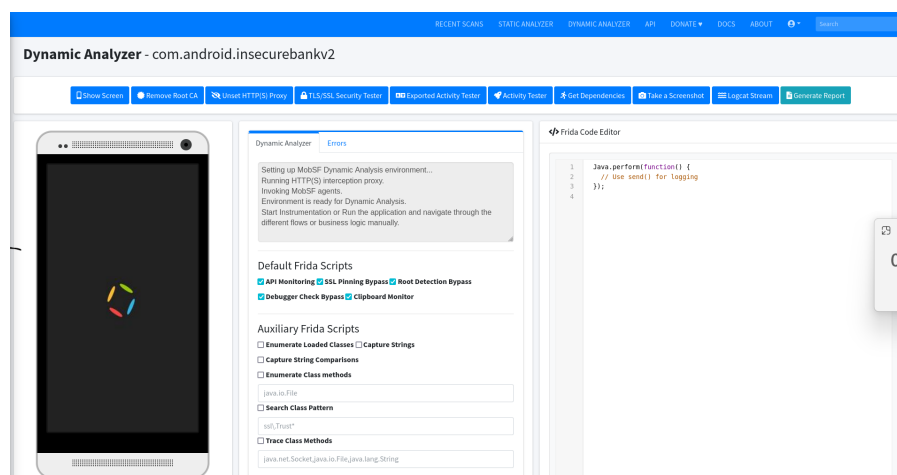


FIGURE 4 – Interface Dynamic Analyzer – environnement prêt

Observation : Le message "Environment is ready for Dynamic Analysis" confirme que le proxy HTTP(S) est actif et que les agents MobSF sont injectés. Scripts Frida actifs : API Monitoring, SSL Pinning Bypass, Root Detection Bypass, Debugger Check Bypass et Clipboard Monitor.

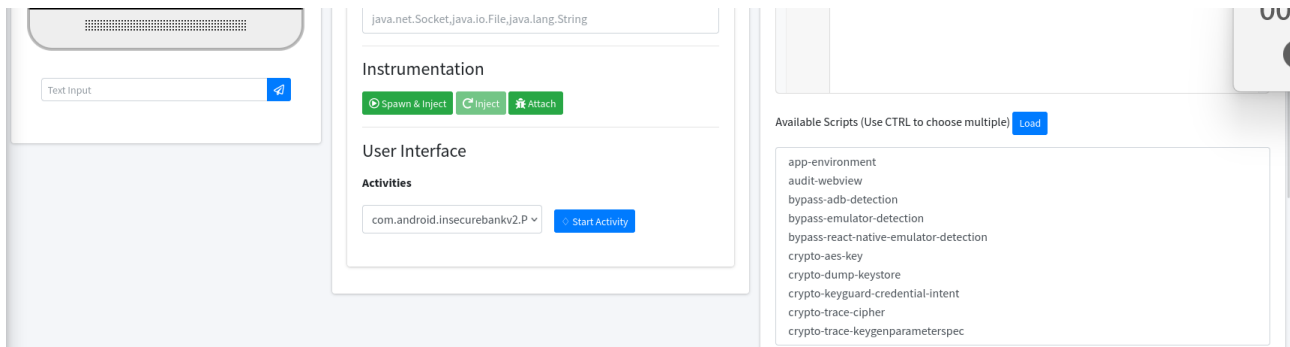


FIGURE 5 – Instrumentation Frida – Spawn & Inject et Shell root actif

Observation : Le prompt `[root@android]#` confirme l'accès root sur l'émulateur. Les scripts Frida disponibles incluent des outils spécialisés comme `crypto-aes-key` et `bypass-emulator-detection` – particulièrement pertinents pour InsecureBankv2.

4.2 Application sur l'émulateur

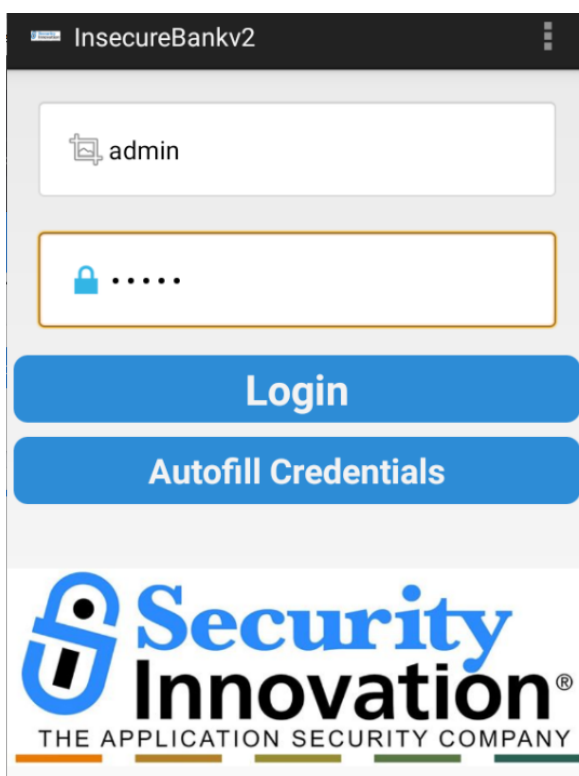


FIGURE 6 – Écran de login avec admin visible

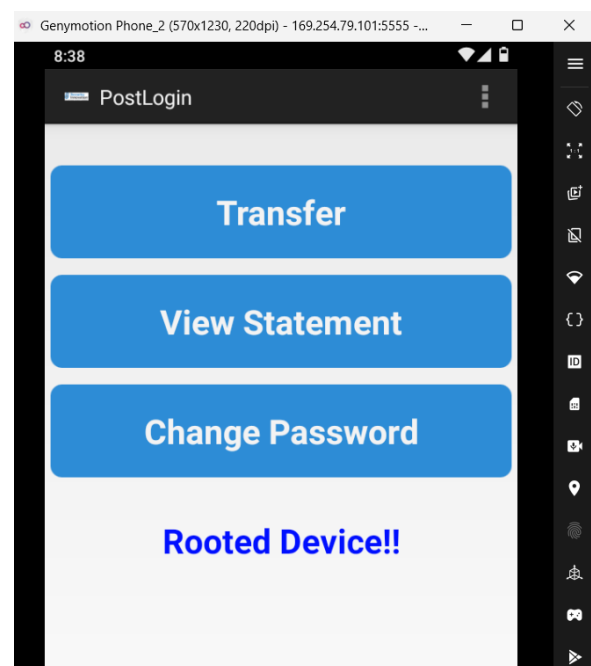


FIGURE 7 – PostLogin – "Rooted Device!!" bypassé

Observation : L'écran login affiche `admin` en clair et propose "Autofill Credentials". L'application affiche "Rooted Device!!" mais Frida bypass cette protection – ce qui démontre l'inefficacité de la détection root seule.

Vulnérabilités identifiées

Constat 1 – Configuration serveur en clair [CRITIQUE]

La configuration du serveur backend (IP et port) est stockée en clair dans les SharedPreferences, accessible à toute application sur l'appareil.

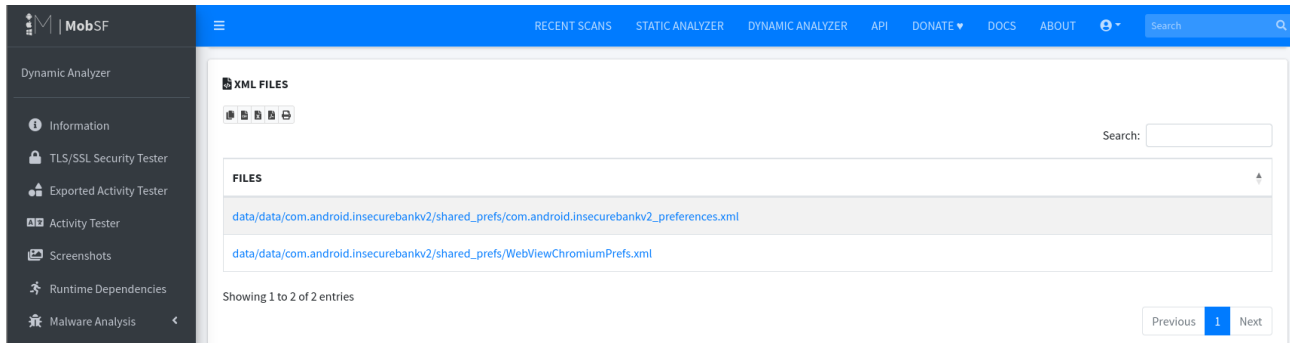


FIGURE 8 – Fichiers XML dans les SharedPreferences

Observation : Deux fichiers XML sont détectés : `insecurebankv2_preferences.xml` et `WebViewChromiumPrefs.xml`. Le premier contient la configuration de connexion au serveur backend en clair.

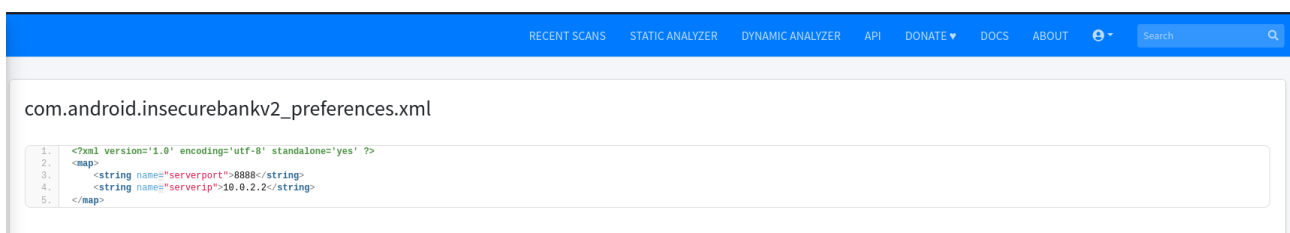


FIGURE 9 – `insecurebankv2_preferences.xml` – `serverip` et `serverport` en clair

Observation : Le fichier preferences révèle `serverip`: 10.0.2.2 et `serverport`: 8888 sans chiffrement. Un attaquant peut extraire ces informations via `adb backup` et rediriger le trafic vers un serveur malveillant.

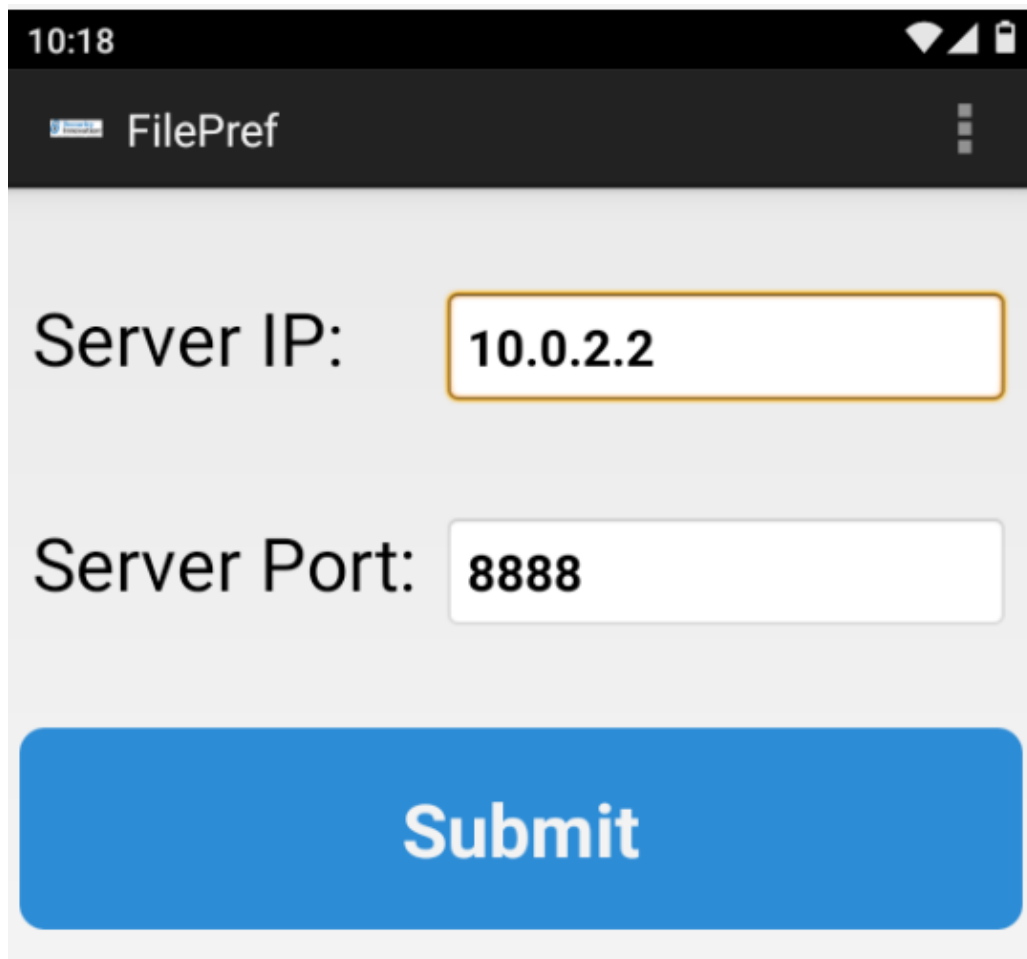


FIGURE 10 – FilePref – IP et port modifiables

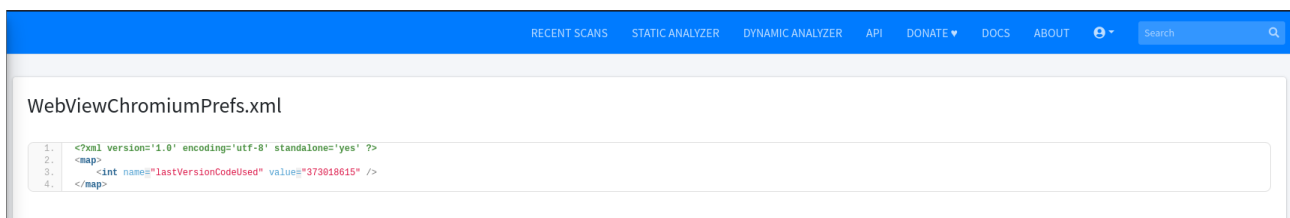


FIGURE 11 – WebViewChromiumPrefs.xml

Observation : L'activity FilePref permet de modifier l'IP et le port du serveur sans authentification – une vulnérabilité critique permettant une attaque Man-in-the-Middle complète.

Remédiation : Stocker la configuration serveur dans Android Keystore et protéger l'activity FilePref par une authentification forte.

Constat 2 – Base de données SQLite non chiffrée [ÉLEVÉE]

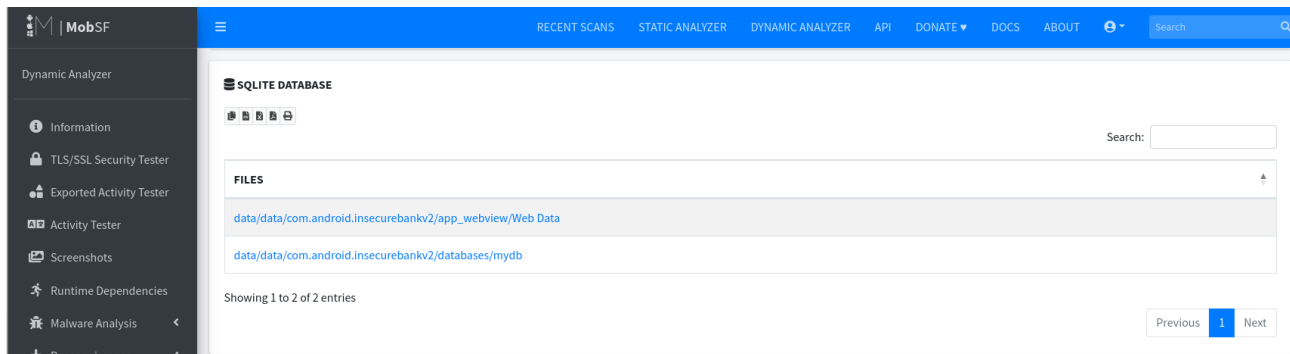


FIGURE 12 – Bases de données SQLite détectées au runtime

Observation : Deux bases de données sont exposées : **Web Data** et **mydb**. Leur accessibilité sans chiffrement permet l'extraction de données sensibles.

meta

key	value
mmap_status	-1
version	81
last_compatible_version	79

autofill

name	value	value_lower	date_created	date_last_used	count
------	-------	-------------	--------------	----------------	-------

credit_cards

guid	name_on_card	expiration_month	expiration_year	card_number_encrypted	date_modified	origin	use_count	use_date	billing_address_id
------	--------------	------------------	-----------------	-----------------------	---------------	--------	-----------	----------	--------------------

autofill_profiles

guid	company_name	street_address	dependent_locality	city	state	zipcode	sorting_code	country_code	date_modified	origin	language_code	use_count	use_date	validity_bitfield	is_client_vali
------	--------------	----------------	--------------------	------	-------	---------	--------------	--------------	---------------	--------	---------------	-----------	----------	-------------------	----------------

autofill_profile_names

guid	first_name	middle_name	last_name	full_name
------	------------	-------------	-----------	-----------

autofill_profile_emails

guid	email
------	-------

FIGURE 13 – Structure de mydb

mydb	
android_metadata	
locale	en_US
names	
id	name
sqlite_sequence	
name	seq

FIGURE 14 – Tables de mydb

Observation : La base mydb contient des tables **names** et **sqlite_sequence** sans chiffrement. En production, ces tables pourraient contenir des noms d'utilisateurs et historiques de transactions.

Remédiation : Utiliser SQLCipher pour chiffrer les bases de données SQLite sensibles.

Constat 3 – Credentials exposés dans les logs [ÉLEVÉE]

```
05-03 10:18:00.099 765 1111 I ActivityTaskManager: START u0 {flag=0x10000000 cmp=com.android.insecurebankv2/ViewStatement} from uid 0/n'
05-03 10:18:05.103 765 2690 I ActivityTaskManager: START u0 {flag=0x10000000 cmp=com.android.insecurebankv2/ChangePassword} from uid 0/n'
05-03 10:18:07.355 765 3187 I ActivityTaskManager: START u0 {flag=0x10000000 cmp=com.android.insecurebankv2/LoginActivity} from uid 0/n'
05-03 10:18:09.958 765 1111 I ActivityTaskManager: START u0 {flag=0x10000000 cmp=com.android.insecurebankv2/com.google.android.gms.ads.AdActivity} from uid 0/n'
05-03 10:18:14.871 765 3187 I ActivityTaskManager: START u0 {flag=0x10000000 cmp=com.android.insecurebankv2/com.google.android.gms.ads.purchase.InAppPurchaseActivity} from uid 0/n'
05-03 10:18:39.031 765 838 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/DoLogin (has extras)} from uid 10104/n'
05-03 10:18:39.227 765 1268 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/FilePrefActivity} from uid 10104/n'
05-03 10:18:39.228 765 1268 W ActivityTaskManager: startActivity called from finishing ActivityRecord {aa4e2b4 u0 com.android.insecurebankv2/DoLogin t19 f}; forcing Intent.FLAG_ACTIVITY_NEW_TASK for: Intent { cmp=com.android.insecurebankv2/FilePrefActivity }/n'
05-03 10:18:42.764 765 838 W NotificationService: Toast already killed. pkg=com.android.insecurebankv2 callback=android.app.ITransientNotification$Stub$Proxy@f026398/n'
05-03 10:18:52.971 765 782 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/DoLogin (has extras)} from uid 10104/n'
05-03 10:18:54.422 765 782 W NotificationService: Toast already killed. pkg=com.android.insecurebankv2 callback=android.app.ITransientNotification$Stub$Proxy@495713c/n'
05-03 10:18:54.760 765 838 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/DoLogin (has extras)} from uid 10104/n'
05-03 10:18:55.804 765 838 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/DoLogin (has extras)} from uid 10104/n'
05-03 10:18:57.161 765 838 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/DoLogin (has extras)} from uid 10104/n'
05-03 10:19:05.947 765 782 I ActivityTaskManager: START u0 {flag=0x10000000 cmp=com.android.insecurebankv2/PostLogin} from uid 0/n'
05-03 10:19:05.979 4070 4070 I .insecurebankv2: type=1400 audit(0.0:165): avc: denied { execute } for name="busybox" dev="sda4" ino=3473 scontext=u:r:untrusted_app_25:s0:c512,c768 tcontext=u:object_r:busybox_exec:s0 tclass=file permissive=1 app=com.android.insecurebankv2/n'
05-03 10:19:05.979 4070 4070 I .insecurebankv2: type=1400 audit(0.0:166): avc: denied { read open } for path="/system/xbin/busybox" dev="sda4" ino=3473 scontext=u:r:untrusted_app_25:s0:c512,c768 tcontext=u:object_r:busybox_exec:s0 tclass=file permissive=1 app=com.android.insecurebankv2/n'
05-03 10:19:05.979 4070 4070 I .insecurebankv2: type=1400 audit(0.0:167): avc: denied { execute no_trans } for path="/system/xbin/busybox" dev="sda4" ino=3473 scontext=u:r:untrusted_app_25:s0:c512,c768 tcontext=u:object_r:busybox_exec:s0 tclass=file permissive=1 app=com.android.insecurebankv2/n'
05-03 10:19:05.979 4070 4070 I which : type=1400 audit(0.0:168): avc: denied { getattr } for path="/system/xbin/busybox" dev="sda4" ino=3473 scontext=u:r:untrusted_app_25:s0:c512,c768 tcontext=u:object_r:busybox_exec:s0 tclass=file permissive=1 app=com.android.insecurebankv2/n'
05-03 10:19:05.983 4070 4070 I which : type=1400 audit(0.0:169): avc: denied { execute } for name="su" dev="sda4" ino=3650 scontext=u:r:untrusted_app_25:s0:c512,c768 tcontext=u:object_r:su_exec:s0 tclass=file permissive=1 app=com.android.insecurebankv2/n'
05-03 10:19:05.983 4070 4070 I which : type=1400 audit(0.0:170): avc: denied { getattr } for path="/system/xbin/su" dev="sda4" ino=3650 scontext=u:r:untrusted_app_25:s0:c512,c768 tcontext=u:object_r:su_exec:s0 tclass=file permissive=1 app=com.android.insecurebankv2/n'
05-03 10:19:14.074 765 782 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/WrongLogin} from uid 10104/n'
05-03 10:19:14.074 765 782 W ActivityTaskManager: startActivity called from non-Activity context; forcing Intent.FLAG_ACTIVITY_NEW_TASK for: Intent { cmp=com.android.insecurebankv2/WrongLogin }/n'
05-03 10:19:14.328 765 1065 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/LoginActivity} from uid 10104/n'
05-03 10:19:17.841 765 782 W NotificationService: Toast already killed. pkg=com.android.insecurebankv2 callback=android.app.ITransientNotification$Stub$Proxy@251b6c7/n'
05-03 10:19:19.862 765 1268 I ActivityTaskManager: START u0 {flag=0x10000000 cmp=com.android.insecurebankv2/ViewStatement} from uid 0/n'
05-03 10:19:28.119 765 1065 I ActivityTaskManager: START u0 {flag=0x10000000 cmp=com.android.insecurebankv2/ViewStatement} from uid 0/n'
05-03 10:19:32.639 765 1065 I ActivityTaskManager: START u0 {flag=0x10000000 cmp=com.android.insecurebankv2/ViewStatement} from uid 0/n'
05-03 10:19:35.351 765 3187 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/WrongLogin} from uid 10104/n'
05-03 10:19:35.352 765 3187 W ActivityTaskManager: startActivity called from non-Activity context; forcing Intent.FLAG_ACTIVITY_NEW_TASK for: Intent { cmp=com.android.insecurebankv2/WrongLogin }/n'
05-03 10:19:35.418 765 1111 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/LoginActivity} from uid 10104/n'
05-03 10:19:38.950 765 782 W NotificationService: Toast already killed. pkg=com.android.insecurebankv2 callback=android.app.ITransientNotification$Stub$Proxy@45a909b/n'
05-03 10:19:56.465 765 1111 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/WrongLogin} from uid 10104/n'
05-03 10:19:56.467 765 1111 W ActivityTaskManager: startActivity called from non-Activity context; forcing Intent.FLAG_ACTIVITY_NEW_TASK for: Intent { cmp=com.android.insecurebankv2/WrongLogin }/n'
05-03 10:19:56.576 765 782 I ActivityTaskManager: START u0 {cmp=com.android.insecurebankv2/LoginActivity} from uid 10104/n'
```

FIGURE 15 – Logcat – Activités et tentatives d'accès root visibles

Observation : Les logs révèlent les appels à **DoLogin**, **FilePrefActivity** et **WrongLogin** en clair. On observe aussi des tentatives d'accès aux binaires **busybox** et **su** refusées par SELinux.

Remédiation : Désactiver les logs en mode production via ProGuard et ne jamais logger d'informations sensibles.

Constat 4 – Fichiers runtime non protégés [ÉLEVÉE]

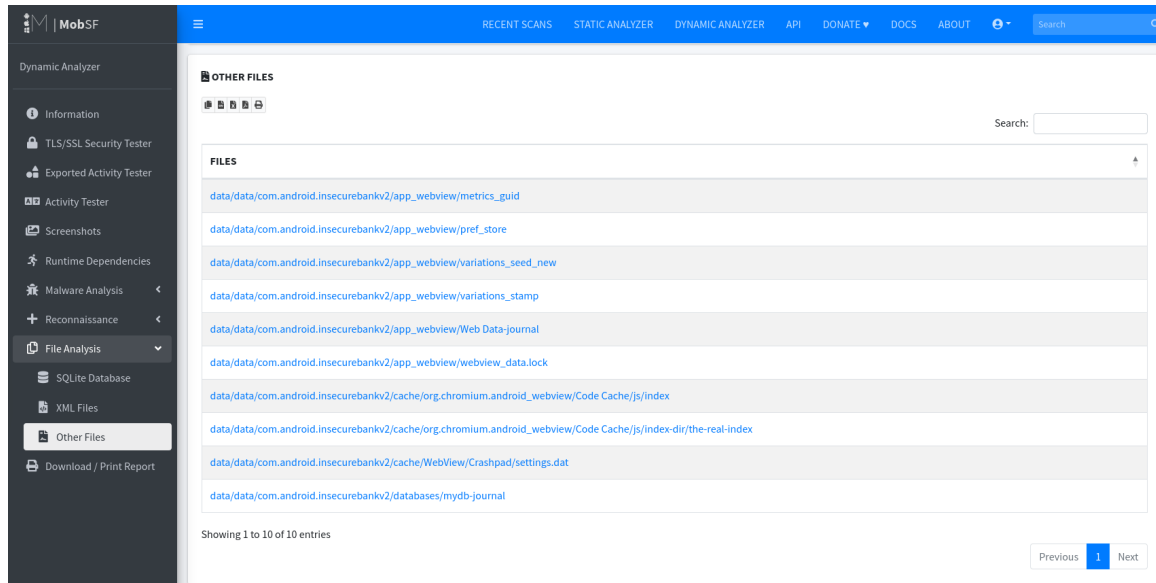


FIGURE 16 – 10 fichiers créés au runtime sans protection

Observation : 10 fichiers sont créés dans `app_webview`, `cache` et `databases` sans chiffrement. La présence de `mydb-journal` indique des transactions de base de données actives et extractibles.

Rapport dynamique MobSF

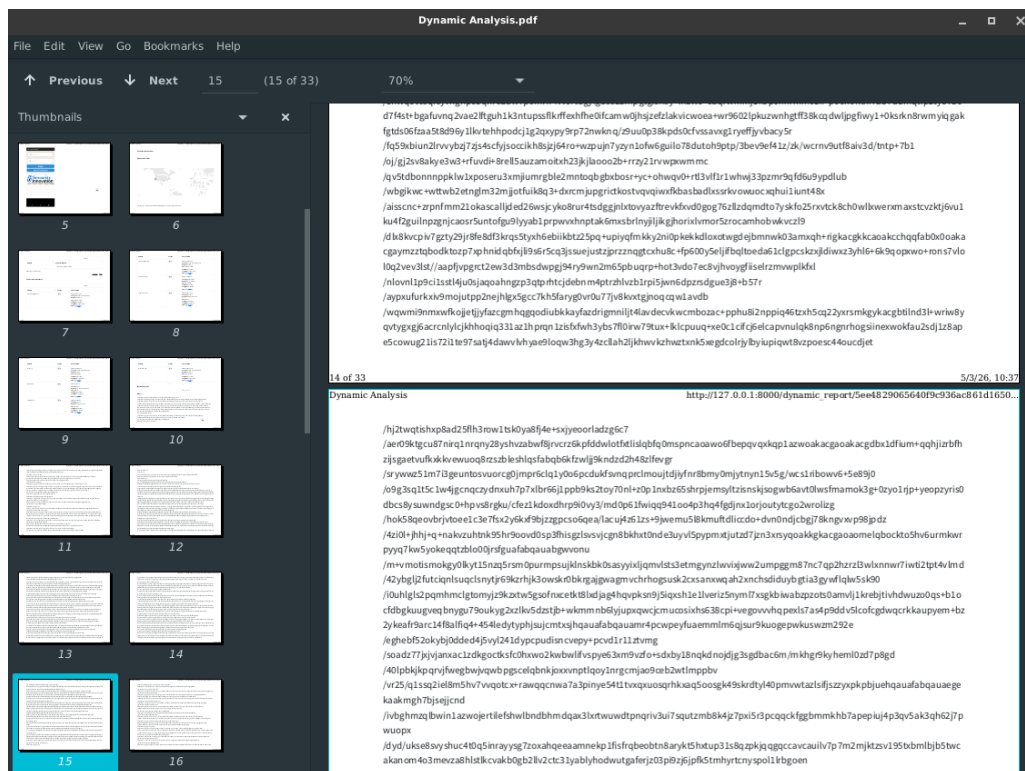


FIGURE 17 – Rapport dynamique MobSF – 33 pages

Observation : Le rapport de 33 pages compile l'ensemble des données collectées pendant l'analyse dynamique, constituant une preuve documentée de toutes les vulnérabilités identifiées lors de l'exécution de l'application.

Synthèse des vulnérabilités

#	Vulnérabilité	Sévérité	Remédiation
1	Configuration serveur en clair	[CRITIQUE]	Android Keystore
2	FilePref modifiable sans auth	[CRITIQUE]	Authentification forte
3	SQLite non chiffrée	[ÉLEVÉE]	SQLCIPHER
4	Credentials dans Logcat	[ÉLEVÉE]	Désactiver logs prod
5	Fichiers runtime non protégés	[ÉLEVÉE]	Chiffrement fichiers

TABLE 2 – Tableau de synthèse des vulnérabilités

Conclusion

L'analyse dynamique d'InsecureBankv2 révèle des vulnérabilités critiques uniquement détectables à l'exécution : configuration serveur en clair, base de données non chiffrée, et credentials dans les logs. MobSF, en combinant le proxy HTTP(S), Frida et la surveillance du système de fichiers, permet d'identifier ces vulnérabilités de manière automatisée. Le score de **28/100** confirme le niveau de risque critique de cette application.